
Tutorial 10

1. A town has f families and k clubs. The i -th club needs a_i members, but can take at most 2 members from any family. Further, each person can belong to at most 1 club. Each family is labelled either *liberal* or *conservative*. We require that at most 70% of the people who belong to conservative families are assigned to a club, and similarly at most 70% of the people who belong to liberal families are assigned to a club. Assume that the j -th family has b_j members, and you are also given whether the j -th family is conservative or liberal.

Show how the problem of assigning people to clubs while satisfying the above constraints can be expressed as a maximum flow problem.

Sol. For each family i , we have a node v_i , and for each club j , we have a node w_j . We have one node called L (for liberal), one node called C (for conservative), and we have a special node s and a node t . We now describe the edges. For each liberal family i , we have an edge (L, v_i) with edge capacity b_i , and for each conservative family i , we have an edge from C to v_i with capacity b_i . For each family i and club j , we have an edge (v_i, w_j) of capacity 2. For each club j , we have an edge (w_j, t) of capacity a_j . We have an edge from s to C and an edge from s to L , each with capacity equal to $0.7N$, where N is the total number of people. We find a max-flow and there is a valid assignment if and only if the value of this flow is equal to $\sum_i a_i$. Indeed, by integrality of max-flow, the flow on an edge (v_i, w_j) is either 0, 1, or 2, and denotes the number of people from family i assigned to club j .

2. A **perfect matching** in a bipartite graph $G = (A \cup B, E)$ with $|A| = |B| = n$ is a matching that saturates every vertex, i.e. a bijection $\sigma : A \rightarrow B$ such that $(a, \sigma(a)) \in E$ for all $a \in A$. Note that a perfect matching may or may not exist in a graph. If it exists, then finding such a matching is the same as finding the maximum matching (which you have seen in the lecture). The following theorem is a classical characterization of existence of a perfect matching in a bipartite graph.

Theorem 1 (Hall's Marriage Theorem). *A bipartite graph $G = (A \cup B, E)$ with $|A| = |B|$ has a perfect matching if and only if for every subset $S \subseteq A$,*

$$|N(S)| \geq |S|,$$

where $N(S) = \{b \in B : (a, b) \in E \text{ for some } a \in S\}$.

Now solve the following.

You are given n borrowers $B_1, \dots, B_n$ and n lenders $L_1, \dots, L_n$. For each pair (B_i, L_j) , you are given the amount borrowed by B_i from L_j (possibly 0), subject to:

- For each i , the total amount borrowed by B_i from all lenders is Rs 100.
- For each j , the total amount lent by L_j to all borrowers is Rs 100.

Find a permutation $\sigma_1, \dots, \sigma_n$ of $\{1, \dots, n\}$ such that B_i has borrowed a strictly positive amount from L_{σ_i} , for each $i = 1, \dots, n$. Show that such a permutation always exists.

Sol. We construct a bipartite graph with the following sets of vertices and edges: on one side we have one vertex for each borrower and on the other side one vertex for each lender. We have an edge between a lender L_j and borrower B_i if the lender has lent a non-zero amount of money to the borrower B_i . Suppose this graph has a perfect matching and matches borrower B_i to lender L_{σ_i} . Then the desired permutation is given by $\sigma_1, \dots, \sigma_n$.

We use Hall's condition to prove that this graph has a perfect matching. Let S be a subset of borrowers and $N(S)$ be the set of neighbours of S in this graph, i.e., the set of lenders which have lent non-zero money to at least one borrower in S . We need to show that $|N(S)| \geq |S|$ for all subsets S . Let x_{ij} denote the money lent to borrower B_i by lender L_j . Then we know that the total money lent to S is $|S|$, i.e.,

$$\sum_{i \in S, j \in N(S)} x_{ij} = |S|.$$

Now, the total money lent by a lender $L_j \in N(S)$ is 1, and hence, the total amount of money lent by L_j to borrowers in S is at most 1, i.e., for any $j \in N(S)$,

$$\sum_{i \in S} x_{ij} \leq 1,$$

and so,

$$\sum_{j \in N(S), i \in S} x_{ij} \leq |N(S)|.$$

Comparing the two equations, we see that $|N(S)| \geq |S|$.

3. You are given an $n \times n$ matrix A with *real* entries. You would like to round each of the entries x in the array to either $\lfloor x \rfloor$ or $\lceil x \rceil$ such that the row and column sums don't change. Give an efficient algorithm which either achieves this rounding or declares that no such rounding is possible. (Hint: Yes, you can solve this using max-flows)
4. Use max-flow/min-cut theorem to answer the following. Suppose in a directed graph G , there are k edge-disjoint paths from s to t and from t to u . Are there k edge disjoint paths from s to u ?

Sol. The answer is YES. Construct a flow network from the given graph by keeping everything else same except adding capacity 1 to each edge.

By the max-flow min-cut theorem, it suffices to show that every cut separating s from u has capacity at least k . Let (A, B) be an arbitrary cut with $s \in A$ and $u \in B$. There are two cases depending on which side t lies.

Case 1: $t \in A$. Since $t \in A$ and $u \in B$, the cut (A, B) is also a valid cut separating t from u . Since there are k edge-disjoint paths from t to u , we have $\text{capacity}(A, B) \geq k$.

Case 2: $t \in B$. Since $s \in A$ and $t \in B$, the cut (A, B) is also a valid cut separating s from t . Since there are k edge-disjoint paths from s to t , we have $\text{capacity}(A, B) \geq k$.

In both cases, every s - u cut has capacity at least k . By the max-flow min-cut theorem, the maximum number of edge-disjoint paths from s to u is at least k .

The rest just follows from integrality of max-flow.

5. Let M be an $n \times n$ matrix with each entry equal to either 0 or 1. Let m_{ij} denote the entry in row i and column j . A *diagonal entry* is one of the form m_{ii} for some i . Swapping rows i and j of the matrix M denotes the following action: we swap the values m_{ik} and m_{jk} for $k = 1, 2, \dots, n$. Swapping two columns is defined analogously. We say that M is *re-arrangeable* if it is possible to swap some of the pairs of rows and some of the pairs of columns (in any sequence) so that after all the swapping, all the diagonal entries of M are equal to 1.
 - (a) Give an example of a matrix M which is **not** re-arrangeable, but for which at least one entry in each row and each column is equal to 1.
 - (b) Give a polynomial-time algorithm that determines whether a matrix M with 0-1 entries is re-arrangeable.

(Hint: Matching)

6. Recall that a set of vertices S in a graph G is a vertex cover if every edge in G has at least one end-point in S . Suppose you have algorithm A which given a graph G and a number k , outputs YES iff G has a vertex cover of size at most k . Assuming that A runs in polynomial time, show that you can find a vertex cover of minimum size in polynomial time.

Sol. First, we can easily find the size of the minimum vertex cover — call it k^* (binary search using the algorithm A). Consider an edge (u, v) . Any solution of size k^* must pick either u or v . In other words, either $G - u$ or $G - v$ should have a vertex cover of size $k^* - 1$. Thus, here is a recursive algorithm A which, given a graph H and a number k , either outputs NO or outputs a vertex cover of size k .

The algorithm $A(H, k)$, where $k = 1$, simply checks if there is a single vertex in H which is a vertex cover. If so, it returns this vertex, otherwise outputs NO. For $k > 1$: let (u, v) be an edge in H . Then we recursively run $A(H - u, k - 1)$ and $A(H - v, k - 1)$. If the answer is NO in both cases, we return NO. Otherwise, say $A(H - u, k - 1)$ returns a set S of size $k - 1$ which is a vertex cover of $H - u$. Then $A(H, k)$ returns $S \cup \{u\}$ (the other case for $H - v$ is similar). Finally, we run A on G, k^* .